`mdarray` design questions and answers

Document #:	P3308R0
Date:	2024/05/21
Project:	Programming Language C++ LEWG

Reply-to:	Mark Hoemmen <mhoemmen@nvidia.com> Christian Trott <crtrott@sandia.gov>

Contents

§ 1 Authors

— Mark Hoemmen (mhoemmen@nvidia.com) (NVIDIA)

— Christian Trott (crtrott@sandia.gov) (Sandia National Laboratories)

§ 2 Revision history

— Revision 0 (pre-St. Louis) to be submitted 2024/05/21

§ 3 Abstract

This paper responds to LEWG's review of P1684R5 ("`mdarray`: An Owning Multidimensional Array Analog of `mdspan`"). It discusses topics that LEWG asked the P1684R5 authors to discuss, and also answers some design questions that the authors have asked themselves. We present this discussion in a way that we hope will help WG21 understand the design motivations of `mdarray`. In order to make this presentation as clear as possible, we have submitted it as a separate paper, rather than as a revision of P1684. We hope that WG21 has the opportunity and interest to read and discuss this paper, and give us feedback to help us improve P1684.

§ 4 Summary of our discussion and conclusions

§ 4.1 Discussions requested by LEWG

LEWG, during its review of P1684R5, asked the authors to consider or discuss the following points.

1. Consider adding a constructor from a (possibly multidimensional) C array

2. Consider adding `in_place_t` constructors (that forward their arguments to the container's constructor), so that `mdarray` can construct the container in place

without a copy or move

3. Discuss construction from initializer lists

§ 4.2 Our conclusions

We think that P1684 should be revised to make the following changes.

1. Add constructors from (possibly multidimensional) C arrays, and corresponding deduction guides, to `mdarray`.

2. Add `in_place_t` constructors and corresponding deduction guides. This would also let users construct `mdarray` from an extents or mapping object, along with a "flat" (one-dimensional) initializer list of values. It would also let us remove wording for *all* the constructors that take `const container_type&` or `container_type&&`.

3. Consider adding multidimensional nested `initializer_list` constructors and deduction guides. This would enable Matlab- or Python-like `mdarray` construction.

§ 4.3 Revisions requested by LEWG

LEWG has also asked for the following revisions:

1. addition of a feature test macro,

2. a description of how to format an `mdarray`, and

3. "the necessary preconditions to avoid use of an object in an invalid state."

We think (2) is best addressed by describing how to format `mdspan`. That would solve a more general problem. A future revision of P1684 will address (1) and (3). We personally would prefer that `mdarray` forbid construction from a moved-from container, and that the only valid `mdarray` operation after moving from that `mdarray` should be destruction.

§ 4.4 Other topics we discuss

1. In order for the `mdarray` constructor that takes `mdspan` to know how to do multidimensional copy in parallel, we should consider adding an `ExecutionPolicy&&` overload to the `mdarray` constructor that takes `mdspan`. On the other hand, adding `in_place_t` constructors to `mdarray` means that if users have a custom container that can be constructed directly from `mdspan` efficiently, then that container would solve any performance issues resulting from constructing an `mdarray` from an `mdspan`.

2. We summarize arguments for and against changing from the R5 container adapter design to a container design, and conclude that the reasons for change do not outweigh the status quo (that is, that `mdarray` should remain a container adapter).

§ 5 Construction from a C array

We have implemented `mdarray` CTAD (constructor template argument deduction) from a possibly multidimensional C array. Here is a pull request that adds the necessary constructor and deduction guide, so you can construct an `mdarray` (with CTAD that deduces `container_type = std::array`) from a possibly multidimensional C array: https://github.com/kokkos/mdspan/pull/329 .

The change includes a new `mdarray` constructor,

```
template<class CArray> requires (
  std::is_array_v<CArray> &&
  std::rank_v<CArray> >= 1u
)
constexpr mdarray(CArray& values)
  : map_(extents_type{}), ctr_{impl::carray_to_array(values)}
{}
```

and a new `mdarray` deduction guide.

```
template<class CArray> requires (
  std::is_array_v<CArray> &&
  std::rank_v<CArray> >= 1u
)
mdarray(CArray& values) -> mdarray<
  std::remove_all_extents_t<CArray>,
  decltype(impl::extents_of_carray(values)),
  layout_right,
  decltype(impl::carray_to_array(values))
>;
```

The new `mdarray` constructor deep-copies the input (as `mdarray` is a container, not a view). Note that `mdspan` has a deduction guide for construction from a one-dimensional C array. However, `mdspan` *cannot* do this for the multidimensional case, because the Standard does not permit conversion of a multidimensional C array to a pointer (or one-dimensional C array). `mdarray` can do this because `mdarray` deep-copies its input array.

§ 6 Add `in_place_t` constructors

§ 6.1 Summary

Adding `in_place_t` constructors that forward arguments to `container_type`'s constructor would let `mdarray` construct its container in place. This would be a truly zero-overhead abstraction for many use cases. The constructors would still need to take an `extents_type` or `mapping_type` parameter as well as the in-place parameters, in order to interpret the flat container as a multidimensional array. Adding these `in_place_t` constructors, along with corresponding deduction guides, could replace the functionality of all the constructors taking `const container_type&` or `container_type&&`, and would

thus let us remove the latter constructors. This would simplify the wording while only slightly increasing verbosity for users.

§ 6.2 Let `mdarray` construct its container

`mdarray` currently constructs its container in two ways. (This ignores construction from `mdspan`, which we will discuss below.)

1. Construction with the number of elements, and optionally one or both of

    a. a single default value with which to fill all of those elements, or

    b. an allocator

2. Copy or move construction of the container itself, possibly also with an allocator

Currently, users who want the container to have a given list of values at construction time must create the container themselves with an initializer list argument, and pass the container into `mdarray`'s constructor. The following example constructs an `mdarray<float, dims<2>, layout_right, std::array<float>>` representing a 2 by 3 matrix.

```
mdarray m{
  extents{2, 3},
  std::array{
    1.0f, 2.0f, 3.0f, // first row
    4.0f, 5.0f, 6.0f  // second row
  }
};
```

Even without CTAD, the constructor's arguments would not change.

```
mdarray<float, dims<2>, std::array<float>> m{
  extents{2, 3},
  std::array{
    1.0f, 2.0f, 3.0f, // first row
    4.0f, 5.0f, 6.0f  // second row
  }
};
```

This is because `mdarray` does not currently have a way for users to construct the container in place, e.g., by passing in values via `initializer_list`. Making the user construct a temporary container poses a performance issue, at least in theory. This issue comes up for all different ways that users might want to construct the container, not just for construction from an initializer list of values. `mdarray` is a container adapter; it permits containers with possibly arbitrary constructors that are not covered by the `vector`-like "size, initial value, and/or allocator" cases.

Adding `in_place_t` constructors to `mdarray` would make it possible for users to construct the container in place in the `mdarray`, with possibly arbitrary arguments. This would be a

guaranteed zero-overhead abstraction, whereas creating a temporary container might not be.

§ 6.3 What would `in_place_t` constructors look like?

We can look at the existing C++ Standard Library classes with `in_place_t` constructors as a model for adding them to `mdarray`. These classes include

— `any`,

— `expected`,

— `optional`, and

— `variant`.

They have in common that they hold at most one object of some type `T`, and they need to construct an instance of `T`. We can look at the most recently added class template `expected` for the general pattern, if we ignore the "`T` is *cv* `void`" case (which doesn't apply to `mdarray`). `expected` has two `in_place_t` constructors.

```
template< class... Args >
constexpr explicit
expected(std::in_place_t,
  Args&&... args);

template< class U, class... Args >
constexpr explicit
expected(std::in_place_t,
  std::initializer_list<U> il,
  Args&&... args);
```

For `mdarray`, the `in_place_t` constructors could not just take the arguments for the container. They would also need to take the extents or mapping, in order to interpret the flat container as a `rank()`-dimensional array. In order not to confuse the extents or mapping with the constructor arguments, we would end up with the following four constructors.

```
template< class... Args >
constexpr explicit mdarray(const extents_type& exts,
  std::in_place_t,
  Args&&... args);

template< class... Args >
constexpr explicit mdarray(const mapping_type& mapping,
  std::in_place_t,
  Args&&... args);

template< class U, class... Args >
constexpr explicit mdarray(const extents_type& exts,
  std::in_place_t,
  std::initializer_list<U> il,
  Args&&... args);
```

```
        template< class U, class... Args >
        constexpr explicit mdarray(const mapping_type& mapping,
          std::in_place_t,
          std::initializer_list<U> il,
          Args&&... args);
```

The first two constructors would require that `container_type` is constructible from `Args...`, while the next two would require that `container_type` is constructible from `std::initializer_list<U>&, Args...`.

This would be a novel use of `in_place_t`. All other Standard Library classes use `in_place_t` for a type that contains at most one thing and needs to construct the thing. `mdarray` contains a layout mapping as well as the container. On the other hand, it's unambiguous that `mdarray` would construct the layout mapping from an `extents_type` or `mapping_type`, so it's reasonable to put that parameter first, before the `in_place_t` parameter.

Note that these `in_place_t` constructors would need corresponding deduction guides in order for CTAD to work as expected. Here (https://godbolt.org/z/MPhEKKdaG) is a brief demo, and here are the required deduction guides.

```
        template<class ValueType, class IndexType, size_t ... Exts, class
ContainerType>
        mdarray(const extents<IndexType, Exts...>&, in_place_t, const
ContainerType&) ->
          mdarray<ValueType, extents<IndexType, Exts...>, layout_right,
ContainerType>;

        template<class ValueType, class IndexType, size_t ... Exts, class
ContainerType>
        mdarray(const extents<IndexType, Exts...>&, in_place_t,
ContainerType&&) ->
          mdarray<ValueType, extents<IndexType, Exts...>, layout_right,
ContainerType>;

        template<class ValueType, class Layout, class IndexType,
std::size_t ... Exts, class ContainerType>
        mdarray(const typename Layout::template mapping<extents<IndexType,
Exts...>>&, in_place_t, const ContainerType&) ->
          mdarray<ValueType, extents<IndexType, Exts...>, Layout,
ContainerType>;

        template<class ValueType, class Layout, class IndexType, size_t
... Exts, class ContainerType>
        mdarray(const typename Layout::template mapping<extents<IndexType,
Exts...>>&, in_place_t, ContainerType&&) ->
          mdarray<ValueType, extents<IndexType, Exts...>, Layout,
ContainerType>;
```

§ 7 Remove `const container_type&` and `container_type&&` constructors

Adding the above `in_place_t` constructors, along with corresponding deduction guides, would let us remove all the constructors taking `const container_type&` or `container_type&&`. The result would make `mdarray` much simpler, while only slightly increasing verbosity for users. For example, the effect of the following constructor in P1684R5

```
        mdarray(const extents_type& exts, container_type&& ctr, const
Alloc& alloc);
```

could be achieved by using the following `in_place_t` constructor instead

```
        mdarray(const extents_type& exts, in_place_t, Args&&...);
```

and passing in the arguments like this.

```
        mdarray< /* template arguments */ > m{exts, in_place,
std::move(ctr), alloc};
```

The constructors of `mdarray` that take `const container_type&` or `container_type&&` exist for a few reasons.

1. They enable CTAD for nondefault containers, such as `array`.
2. They enable copying the values.
3. They pass along the container's state, including any allocator.
4. The constructors that move-construct the container allow using a container as a kind of allocator for `mdarray`, that can be passed along via `mdarray`'s `extract_container` member function.

Use case (1) is handy, especially for small arrays. It only requires the constructors taking `container_type&&`, because the typical use case is to construct the container in place as an `mdarray` constructor argument, like this.

```
        mdarray m{
          extents{2, 3},
          array{
            1.0f, 2.0f, 3.0f, // first row
            4.0f, 5.0f, 6.0f  // second row
          }
        };
```

Replacing the `const extents_type&, container_type&&` constructor with a `const extents_type&, in_place_t, Args&&...` constructor, and adding corresponding deduction guides, would only make this a bit more verbose.

```
        mdarray m{
          extents{2, 3}, in_place,
          array{
            1.0f, 2.0f, 3.0f, // first row
            4.0f, 5.0f, 6.0f  // second row
          }
        };
```

Adding the `in_place_t` constructors would satisfy use cases (2) and (3) for any containers with a reasonable set of constructors and access to members. Here is an example of use case (4), the "passing along storage via container" use case that is the main intended purpose of `extract_container` (see https://godbolt.org/z/MvxceW7rs for a quick demo). The `in_place_t` constructor handles this use case as well, with only a bit more verbosity.

```
        template<class ValueType, class Extents>
        std::vector<ValueType> use_vector(const Extents& exts,
std::vector<ValueType>&& x) {
          using std::layout_right;
          using container_type = std::vector<float>;
          using extents_type = Extents;
          using mdarray_type = mdarray<ValueType, extents_type,
layout_right, container_type>;

          std::cout << "use_vector: x.size() on input: " << x.size()
            << "; x.data(): " << x.data() << "\n";

          layout_right::mapping mapping{exts};
          const auto size = mapping.required_span_size();
          std::cout << "  required_span_size: " << size << "\n";
          if (static_cast<std::size_t>(size) > x.size()) {
            x.resize(size);
          }
          mdarray_type x_md(mapping, std::in_place, std::move(x));
          return std::move(x_md).extract_container();
        }

        int main() {
          using stdex::extents;
          std::vector<float> storage(6);
          extents exts0{2, 3};
          extents exts1{3, 4};
          extents exts2{2, 2};
          auto y = use_vector(exts2, use_vector(exts1, use_vector(exts0,
std::move(storage))));

          std::cout << "size of result of use_vector: " << y.size() <<
"\n";
          assert(y.size() == 12u);

          return 0;
        }
```

§ 8 Do not add separate constructors from “flat” initializer list

The previous section leads naturally to the discussion of `initializer_list` constructors. A LEWG member, during LEWG’s 2023/11/09 review of P1684R5, asked whether we support use of `initializer_list` in constructors to specify the `mdarray`’s values. We presume that this means a flat `initializer_list<U>` with `U` convertible to `value_type`. Since this list is flat, the constructor would need information about the extents and rank as well. If we ignore CTAD, then the `mdarray` type would need all static extents, like this.

```
mdarray<float, extents<int, 2, 3>> m {
  1.0f, 2.0f, 3.0f,
  4.0f, 5.0f, 6.0f
};
```

With any dynamic extents, or with CTAD, the user would need to pass in an extents or mapping object. It would make sense to put the values in an `initializer_list`, in order to separate them from the extents or mapping object.

```
mdarray m {
  dextents<int, 2>{2, 3},
  {
    1.0f, 2.0f, 3.0f,
    4.0f, 5.0f, 6.0f
  }
};
```

The resulting constructors would look like this.

```
template< class U, class... Args >
constexpr explicit
expected(const extents_type& exts,
  std::initializer_list<value_type> il);
template< class U, class... Args >
constexpr explicit
expected(const mapping_type& mapping,
  std::initializer_list<value_type> il);
```

That looks almost the same as the `in_place_t` constructors presented in the previous section. The only difference would be semantic. Constructors taking `in_place_t` would construct the container with the arguments – whatever that means for the specific container type. For example, the arguments might include an allocator. In contrast, the above non-`in_place_t` constructors would fill the container with the values in the `initializer_list`, for any container type. This would add to the container type’s requirements, that forwarding the `initializer_list` to the container would construct it with those values. There are no existing named container requirements in the Standard that would express this requirement; we would need to come up with new wording. In contrast, the `in_place_t` constructors would merely forward directly to the container’s constructor,

and thus would not impose any requirements intrinsically. On the other hand, users would need to know what arguments make sense for the container types they use.

If we only add the `in_place_t` constructors, that would make CTAD use cases slightly more verbose, but it would make both the wording and implementation easier.

```
mdarray m {
  extents{2, 3},
  in_place,
  {
    1.0f, 2.0f, 3.0f,
    4.0f, 5.0f, 6.0f
  }
};
```

There's another concern here, which is that `initializer_list`'s `size()` function can't be used in a deduction guide. Thus, with CTAD, `initializer_list` construction would *necessarily* result in `std::vector` being the container type, instead of `std::array`. The user knows everything at compile time – they have spelled out for us all the values with which to construct the array! – but we would end up throwing that out, because of a limitation of `initializer_list`. Now we're calling `new` for an array of 6 values that could very well be a compile-time constant.

Contrast this with the current (R5) approach. The following example takes four *fewer* characters than the `in_place` example above, but it always uses `std::array`.

```
mdarray m {
  extents{2, 3},
  array{
    1.0f, 2.0f, 3.0f,
    4.0f, 5.0f, 6.0f
  }
};
```

What does this tell us?

1. Flat `initializer_list` plus CTAD is, surprisingly, *not* a zero-overhead abstraction, because it forces use of the default container `std::vector`.

2. Flat `initializer_list` *without* CTAD could be a zero-overhead abstraction. However, this imposes an additional requirement on the container type, which is both syntactic (constructible from an `initializer_list`) and semantic (construction from an `initializer_list` fills it with those values in that order). We don't have existing wording in the Standard to reuse for this.

3. The `in_place_t` constructors cover both use cases. They also impose fewer requirements on the container (or rather, they push meeting those requirements to the user's code). In addition, they let users pass in arguments *other* than the initial values (e.g., an allocator) to the container type's constructor.

We conclude that the `in_place_t` constructors are worth adding, while separate non-`in_place_t` `initializer_list` constructors would *not* be worth adding.

§ 9 Consider adding construction from nested initializer list

§ 9.1 Why we want this

It would be attractive if `mdarray` could take a nested initializer list of values, and automatically deduce its (compile-time) rank and (run-time) extents. For example, the following should deduce the extents as `dims<2>`, that is, `extents<size_t, dynamic_extent, dynamic_extent>`.

```
mdarray m_2d{
  {1.0f, 2.0f, 3.0f},
  {4.0f, 5.0f, 6.0f}
};
```

Deeper nesting can introduce rank-3 or even higher-rank `mdarray`. This feature would make `mdarray` construction look familiar to users of Matlab or Python, both popular languages for machine learning and numerical computations. It's idiomatic for users of those languages to construct multidimensional arrays with their values and extents all at once. Here's a Matlab construction of a rank-2 array with 3 rows and 4 columns,

```
arr = [
  1,  2,  3,  4;
  5,  6,  7,  8;
  9, 10, 11, 12
]
```

and here's the equivalent Python construction (using NumPy).

```
arr = np.array([
  [1,  2,  3,  4],
  [5,  6,  7,  8],
  [9, 10, 11, 12]
])
```

Users would want to construct a rank-2 `mdarray` in the same way.

```
mdarray arr {
  {1, 2, 3, 4},
  {5, 6, 7, 8},
  {9, 10, 11, 12}
};
```

§ 9.2 Limits of this approach

1. The extents would need to be run-time values, since `initializer_list` (unlike `array` or `span`) doesn't encode its `size()` in the type. (That's too bad, because the user has already told the compiler how long each initializer list is!)

2. Nesting means that the implementation would need to traverse the inner lists (to one less than the nesting level) to count the number of elements, allocate the container with the count, and only then fill the container by traversing the input again. This fills the container twice, and requires that `value_type` be default constructible.

3. The constructor would have a precondition that all the inner lists at the same level would have the same size. (That precondition could be checked at run time.)

§ 9.3 Alternative: `make_mdarray` function

As an alternative to CTAD and constructors, one could imagine a `make_mdarray<ValueType, Level>` function template with overloads for different levels of `initializer_list` nesting. We would prefer CTAD, though, because it reduces the number of names to remember. Users who want to be more explicit about types can always spell out `mdarray`'s template arguments.

§ 9.4 Implementation approach

We can accomplish this with a single `mdarray` constructor, by introducing a type alias (which in the wording would be exposition only) to express nested `initializer_list` with a known level of nesting.

```
        namespace impl {

        template<class T, std::size_t Level>
        struct init_list {
          static_assert(Level != 0u);
          using type = std::initializer_list<typename init_list<T, Level -
1u>::type>;
        };

        template<class T>
        struct init_list<T, 0> {
          using type = T;
        };

        }
```

This would let `mdarray` have a constructor like the following.

```
        mdarray(typename init_list<value_type, extents_type::rank()>::type
values)
          requires(Rank != 0);
```

Here is a demo: https://godbolt.org/z/EKc3eaafb . The only issue is that we would need a deduction guide for each level of nesting (and thus for each rank), starting with 1 and

going up to some implementation-defined limit on the rank. `mdarray` could still be constructed and used with higher rank, but CTAD from nested `initializer_list` would not work for them. Here are the new deduction guides we would need.

```
template<class ValueType>
  requires(not impl::is_initializer_list_v<ValueType>)
mdarray(std::initializer_list<ValueType>)
  -> mdarray<ValueType, dims<1>>;

template<class ValueType>
  requires(not impl::is_initializer_list_v<ValueType>)
mdarray(
  std::initializer_list<
    std::initializer_list<ValueType>
  >)
  -> mdarray<ValueType, dims<2>>;

template<class ValueType>
  requires(not impl::is_initializer_list_v<ValueType>)
mdarray(
  std::initializer_list<
    std::initializer_list<
      std::initializer_list<ValueType>
    >
  >)
  -> mdarray<ValueType, dims<3>;

// ... and so on, up to some implementation limit number of ranks
```

As far as we know, there's no generic way to define one (or a constant number of) deduction guide(s) for all the levels of nesting. We would welcome clever suggestions for fixing that.

This approach would only makes the notation more concise in the CTAD case. If users must name the extents type anyway, then it's an occasion for error to specify the extents in two places – as the extents type template argument, and implicitly in the lengths of the initializer lists.

§ 10 Add an `ExecutionPolicy&&` overload to construction from `mdspan`

`mdarray` currently has a constructor from `mdspan`, that deep-copies the elements of the `mdspan`. This constructor is the only way for users to construct an `mdarray` that is a deep copy of an arbitrary `mdspan`. However, this constructor introduces a potential performance problem. All the other constructors that copy the elements of their input, copy the container directly. This can rely on whatever optimizations the container has, including copying in parallel and/or using an accelerator. The constructor from `mdspan` must access the elements directly, in generic code. It has no way to deduce the correct execution policy in order to make copying parallel, for example. This has led to implementation divergence,

for example in NVIDIA's RAPIDS RAFT library, which depends on the ability to use CUDA streams to dispatch allocations and copying operations.

A natural fix would be to add a constructor with two parameters, `ExecutionPolicy&&` and `mdspan`.

```
        template<class OtherElementType, class OtherExtents,
                 class OtherLayoutPolicy, class Accessor>
          explicit(/* see below */)
          constexpr mdarray(ExecutionPolicy&& policy,
            const mdspan<OtherElementType, OtherExtents,
              OtherLayoutPolicy, Accessor>& other);
```

While the Standard currently offers no generic way to use the `ExecutionPolicy` for copying from an `mdspan` into a container in parallel, this would at least offer a hook for implementations to optimize inside the constructor. Users who call this constructor would assert that it is correct to copy in parallel from the input `mdspan` into the `mdarray`'s container.

P3240R0 (Copy and fill for `mdspan`) would provide a copy algorithm with an `ExecutionPolicy&&` overload that copies from the elements of a source `mdspan` to a destination `mdspan`. This would let `mdarray`'s wording (not just its implementation) specify how copying happens. However, P3240R0 requires that all the elements of the destination of the copy have started their lifetimes. For generic element types, this would force `mdarray` to allocate and fill storage first, before copying. For implicit-lifetime types (including all arithmetic types – a common case for use of `mdarray`), a custom container (not `std::vector`) could allocate without filling. Then, `mdarray`'s constructor could copy from the input `mdspan` to a temporary `mdspan` viewing its container's elements.

This leaves non-implicit-lifetime types pessimized. On the other hand, adding `in_place_t` constructors to `mdarray` means that if users have a custom container that can be constructed directly from `mdspan` efficiently, then that container would solve any performance issues resulting from constructing an `mdarray` from an `mdspan`. That, plus adding an `ExecutionPolicy&&`, `mdspan` constructor to `mdarray`, should address any performance issues.

§ 11 Why `mdarray` is a container adapter and not a container

§ 11.1 Consistency with `mdspan`

`mdarray` is a container adapter. It has a `ContainerType` template parameter, stores an instance of `ContainerType`, and returns references that it gets from the container's `operator[]` member function. The discussions in the previous sections presume this design.

The main reason we chose this approach is for consistency with `mdspan`. `mdspan` imposes multidimensional behavior on a flat *view* of elements. Thus, by analogy, `mdarray` should express multidimensional behavior on a flat *container* of elements. That implies a container adapter design. We could then say that "`mdspan` is a view adapter; `mdarray` is a container adapter." The R0 design reflected this even more explicitly, with its `ContainerPolicy` template parameter analogous to `mdspan`'s `AccessorPolicy`.

In order to understand that design choice more fully, one should entertain the alternative of making `mdarray` a container. The discussion below explains what that design would look like, and its advantages and disadvantages over the current container adapter design.

§ 11.2 What would a container design look like?

A container design with the same functionality as `mdarray` in P1684R5 would actually need two containers, which we provisionally call `md_fixed_array` and `md_dynamic_array`.

1. `md_fixed_array`
   - Stores all elements as if in a `std::array`
   - Requires that all the extents be static
   - Move behavior is like that of `std::array`
2. `md_dynamic_array`
   - Stores all elements as if in a dynamically allocated container, like `std::vector` (but without resizing)
   - Permits any combination of dynamic or static extents (including all static extents)
   - Move behavior is like that of `std::vector`: constant time, does not copy the elements, and leaves the moved-from container empty; the moved-from container's elements can no longer be accessed
   - Has all the allocator-aware machinery of `std::vector`

§ 11.3 Advantages of a container design

1. It would solve the problem of defining the moved-from state of `mdarray`, since we could define it ourselves.
2. `mdarray` exists entirely to make `mdspan` easier to use for common cases. We expect that most users would not use custom container types with `mdarray`, for example.
3. `mdarray` needs preconditions for corner cases, such as having one or more dynamic extents, but with `array` as its container type. `md_fixed_array` and `md_dynamic_array` would not need this. (Note that in previous P1684 reviews,

LEWG explicitly rejected our proposal to make `mdarray`'s default container type `array` if all the extents are static, and `vector` otherwise.)

4. Separating the `md_fixed_array` and `md_dynamic_array` cases would simplify wording for each.

5. The existing Container requirements in the Standard do not cover what `mdarray` needs. `mdarray` thus finds itself forging new wording ground. This would be in a part of the Standard – container requirements – about which we have heard WG21 members express concerns: for example, that it's old, that it scatters requirements in many different places (making it hard to maintain), and that it is not as consistent as we wish it could be.

6. The most reasonable default container type for a container adapter is `vector`. However, this is not a zero-overhead abstraction, as `vector` stores a capacity in order to support efficient resizing, but `mdarray` cannot be resized and thus does not need the capacity. The Standard does not have a container type like that.

Regarding the default container type for `mdarray`, one might think of `dynarray`. (N3662 ("C++ Dynamic Arrays") proposed `dynarray`. It was later voted out of C++14 into a Technical Specification.) However, `dynarray` is not quite the container needed here. It does not have move construction at all, and thus cannot promise anything about the cost or postconditions of moves.

§ 11.4 Disadvantages of a container design

1. The Standard Library's container requirements all include iterators. For `mdarray`-like container classes, this would require us to define iterators. We deliberately did not define iterators for `mdspan`, because they are nearly impossible to make performant without fanciful compiler support.

2. The container approach would not solve the problem of how to define the behavior of a moved-from object. Consider `md_dynamic_array`. It's tempting to set its dynamic extents to zero after moving from the object, so that the moved-from object has zero elements. However, any static extents could not be changed. If the object has all static extents, it would still have a nonzero number of elements. It would also be confusing for an `md_dynamic_array` object with dynamic extents to behave differently at run time than an `md_dynamic_array` object with static extents.

3. The container adapter approach gives users a hook to specify how copying elements happens in parallel (with the exception of assignment from `mdspan`; see section below). For example, a custom container might have an accelerator-specific resource (e.g., a CUDA stream) in it that would be used for copies. The container approach would make this impossible to specify in a generic way; users would have no place other than a custom allocator to store parallel execution information, but generic C++ code wouldn't have a way to get that information out and pass it into `std::copy` (for example).

4. Specifying a new container type is complicated. We would find ourselves replicating a lot of `vector`'s wording. As a result, total wording length could actually increase. This would also impose future costs if WG21 later wants to revise `vector`'s wording.

5. Construction from `container_type&&` and the member function `container_type&& extract_container()` support a specific use case: representing dynamically allocated storage as a container and "passing it along" a chain of operations. Changing from a container adapter to a container would make this use case harder. Users would need to create an allocator, instead of just creating a `vector` and passing it along.

§ 11.5 No major notational advantages

All this being said, though, the container approach offers no major notational advantages for users over the container adapter approach. For example, the P1684R5 container adapter design permits the following CTAD construction.

```
mdarray m {
  extents{2, 3},
  array{
    1.0f, 2.0f, 3.0f,
    4.0f, 5.0f, 6.0f
  }
};
```

The natural analog with `md_fixed_array` would be an `initializer_list<value_type>` constructor (without `in_place_t`, which would only make sense for a container adapter).

```
mdarray_fixed_array m {
  extents{2, 3},
  {
    1.0f, 2.0f, 3.0f,
    4.0f, 5.0f, 6.0f
  }
};
```